```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score


plt.style.use('seaborn-v0_8-whitegrid')


print("Loading Adult Income dataset from OpenML...")
adult = fetch_openml(data_id=1590, as_frame=True, parser='auto')
df = adult.frame.drop(columns=['fnlwgt'], errors='ignore')


target_col = 'class' if 'class' in df.columns else 'income'
y = df[target_col].apply(lambda x: 1 if '>50K' in str(x) else 0)
X = df.drop(columns=[target_col])


num_cols = X.select_dtypes(include=['int64',
'float64']).columns.tolist()
cat_cols = X.select_dtypes(include=['object',
'category']).columns.tolist()


num_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', num_transformer, num_cols),
    ('cat', cat_transformer, cat_cols)
```

```
])


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


lr_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])

dt_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', DecisionTreeClassifier(max_depth=3,
random_state=42)) # max_depth=3 for clear plot visualization
])

print("Training Logistic Regression and Decision Tree models...")
lr_pipeline.fit(X_train, y_train)
dt_pipeline.fit(X_train, y_train)


def evaluate_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    return [
        accuracy_score(y_test, y_pred),
        precision_score(y_test, y_pred),
        recall_score(y_test, y_pred),
        f1_score(y_test, y_pred)
    ]

metrics_list = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
lr_scores = evaluate_model(lr_pipeline, X_test, y_test)
dt_scores = evaluate_model(dt_pipeline, X_test, y_test)

comparison_df = pd.DataFrame({
    'Metric': metrics_list,
    'Logistic Regression': lr_scores,
    'Decision Tree (depth=3)': dt_scores
})

print("\n" + "="*50)
print("       MODEL PERFORMANCE COMPARISON TABLE")
print("="*50)
print(comparison_df.to_string(index=False))
print("="*50 + "\n")
```

```
metrics_df = pd.DataFrame({
    'Metric': metrics_list * 2,
    'Score': lr_scores + dt_scores,
    'Model': ['Logistic Regression'] * 4 + ['Decision Tree'] * 4
})

plt.figure(figsize=(10, 6))
ax = sns.barplot(data=metrics_df, x='Metric', y='Score', hue='Model',
palette='Set2')
plt.title('Performance Comparison: Logistic Regression vs Decision
Tree', fontsize=14, fontweight='bold')
plt.ylim(0, 1.0)
plt.ylabel('Score')

for p in ax.patches:
    if p.get_height() > 0:
        ax.annotate(f'{p.get_height():.4f}',
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha='center', va='bottom', fontsize=10, xytext=(0,
3),
                    textcoords='offset points')

plt.tight_layout()
plt.show()


feature_names = num_cols + list(
    lr_pipeline.named_steps['preprocessor']
    .named_transformers_['cat']
    .named_steps['onehot']
    .get_feature_names_out(cat_cols)
)


plt.figure(figsize=(20, 10))
plot_tree(
    dt_pipeline.named_steps['classifier'],
    feature_names=feature_names,
    class_names=['<=50K', '>50K'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title('Decision Tree Structure (max_depth=3)', fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.show()
```

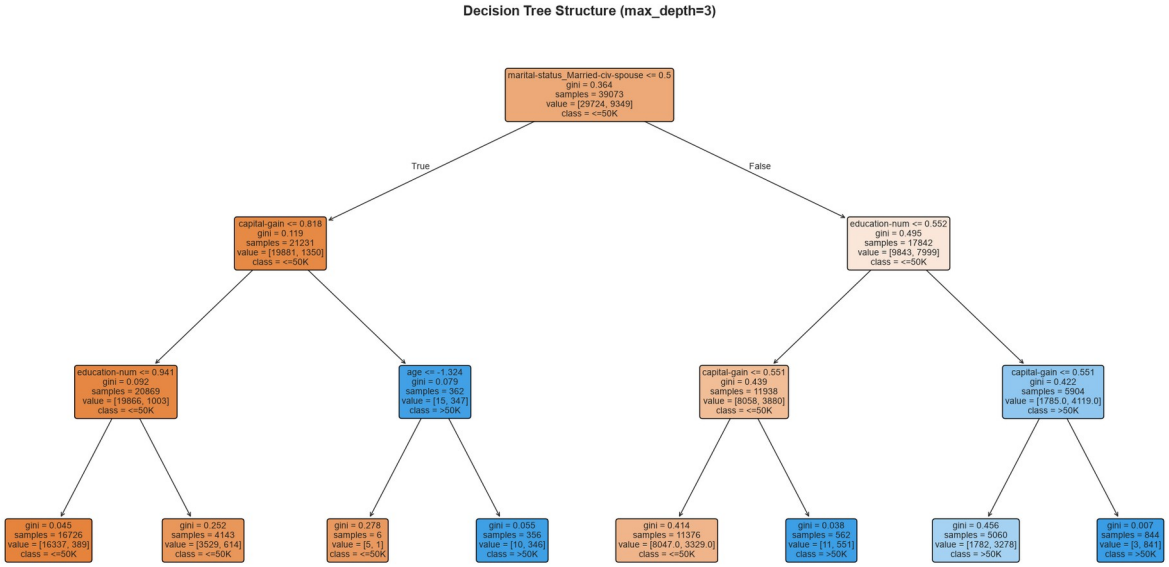
```
coefs = pd.Series(lr_pipeline.named_steps['classifier'].coef_[0],
index=feature_names)
top_pos = coefs.nlargest(8)
top_neg = coefs.nsmallest(8)
top_coefs = pd.concat([top_pos, top_neg]).sort_values()

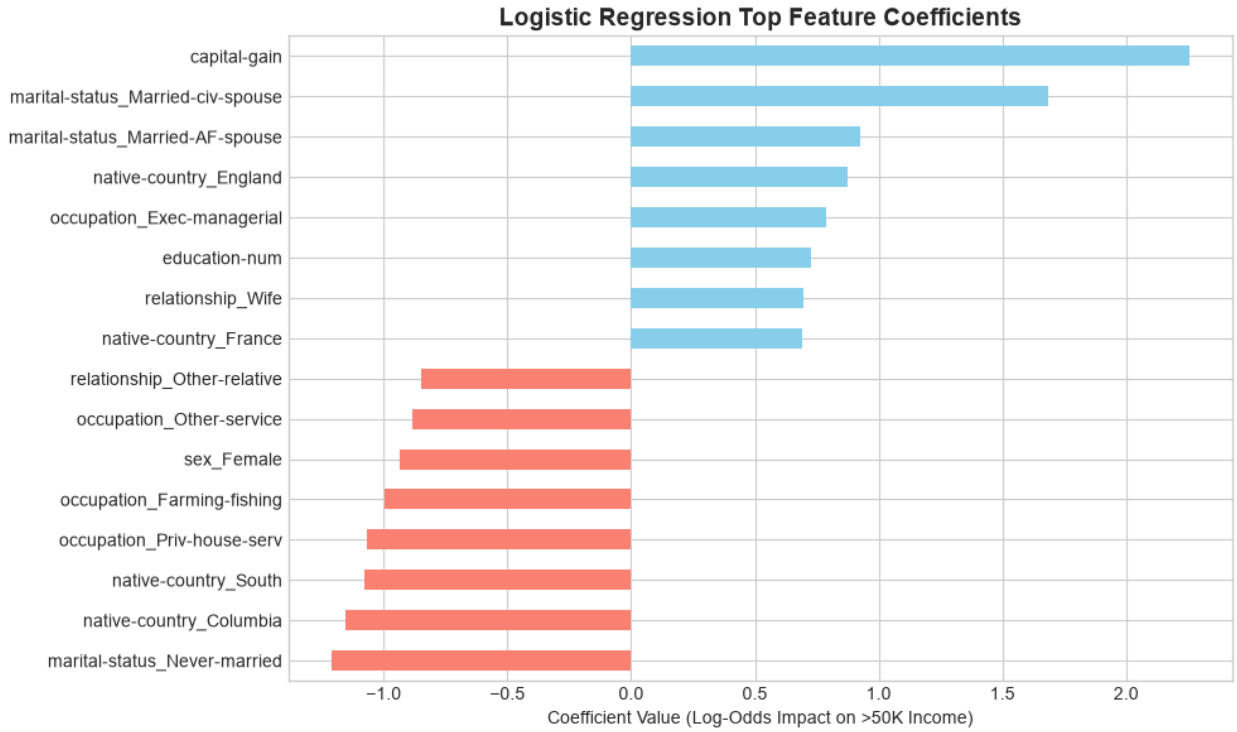
plt.figure(figsize=(10, 6))
top_coefs.plot(kind='barh', color=np.where(top_coefs > 0, 'skyblue',
'salmon'))
plt.title('Logistic Regression Top Feature Coefficients', fontsize=14,
fontweight='bold')
plt.xlabel('Coefficient Value (Log-Odds Impact on >50K Income)')
plt.tight_layout()
plt.show()
```

```
Loading Adult Income dataset from OpenML...
Training Logistic Regression and Decision Tree models...

==================================================
       MODEL PERFORMANCE COMPARISON TABLE
==================================================
   Metric  Logistic Regression  Decision Tree (depth=3)
 Accuracy             0.853004                 0.845532
Precision             0.742213                 0.742824
   Recall             0.591104                 0.542344
 F1-Score             0.658095                 0.626947
==================================================
```


Performance Comparison: Logistic Regression vs Decision Tree
Model
Logistic Regression
Decision Tree
Score
Metric
Accuracy
Precision
Recall
F1-Score
0.8530
0.8455
0.7422
0.7428
0.5911
0.5423
0.6581
0.6269
1.0
0.8
0.6
0.4
0.2
0.0


Decision Tree Structure (max_depth=3)
marital-status_Married-civ-spouse <= 0.5
gini = 0.364
samples = 39073
value = [29724, 9349]
class = <=50K
True
False
capital-gain <= 0.818
gini = 0.119
samples = 21231
value = [19881, 1350]
class = <=50K
education-num <= 0.552
gini = 0.495
samples = 17842
value = [9843, 7999]
class = <=50K
education-num <= 0.941
gini = 0.092
samples = 20869
value = [19866, 1003]
class = <=50K
age <= -1.324
gini = 0.079
samples = 362
value = [15, 347]
class = >50K
capital-gain <= 0.551
gini = 0.439
samples = 11938
value = [8058, 3880]
class = <=50K
capital-gain <= 0.551
gini = 0.422
samples = 5904
value = [1785.0, 4119.0]
class = >50K
gini = 0.045
samples = 16726
value = [16337, 389]
class = <=50K
gini = 0.252
samples = 4143
value = [3529, 614]
class = <=50K
gini = 0.278
samples = 6
value = [5, 1]
class = <=50K
gini = 0.055
samples = 356
value = [10, 346]
class = >50K
gini = 0.414
samples = 11376
value = [8047.0, 3329.0]
class = <=50K
gini = 0.038
samples = 562
value = [11, 551]
class = >50K
gini = 0.456
samples = 5060
value = [1782, 3278]
class = >50K
gini = 0.007
samples = 844
value = [3, 841]
class = >50K


Logistic Regression Top Feature Coefficients
capital-gain
marital-status_Married-civ-spouse
marital-status_Married-AF-spouse
native-country_England
occupation_Exec-managerial
education-num
relationship_Wife
native-country_France
relationship_Other-relative
occupation_Other-service
sex_Female
occupation_Farming-fishing
occupation_Priv-house-serv
native-country_South
native-country_Columbia
marital-status_Never-married
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
Coefficient Value (Log-Odds Impact on >50K Income)